UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

GUILHERME DOMINGOS SANTANA

Aprimoramento de SLM para Geração de Código Paralelo em Computação de Alto Desempenho

Monografia apresentada como requisito parcial para a obtenção do grau de Bacharel em Engenharia da Computação

Orientador: Prof. Dr. Arthur Lorenzon

Porto Alegre
2026

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Reitora: Profª. Marcia Barbosa
Vice-Reitor: Prof. Pedro Costa
Pró-Reitor de Graduação: Prof. Vladimir Pinheiro do Nascimento
Diretor do Instituto de Informática: Prof. Luciano Paschoal Gaspary
Coordenador do Curso de Engenharia de Computação: Prof. Cláudio Machado Diniz
Bibliotecário-chefe do Instituto de Informática: Alexsander Borges Ribeiro

AGRADECIMENTOS

Quero iniciar meus agradecimentos dedicando este trabalho à minha mãe, que infelizmente perdi ao longo dos meus sete anos e meio de graduação. Sei que ela estaria profundamente orgulhosa deste momento, e sua memória permanece sendo minha maior inspiração.

Agradeço também ao meu pai e a toda a minha família, que sempre estiveram ao meu lado, oferecendo apoio e suporte em cada decisão que precisei tomar. Estendo meus agradecimentos aos meus amigos, que foram presença constante nos momentos de descontração e alívio ao longo desta jornada. Sou igualmente grato à minha namorada pelo amor, carinho e suporte que foram essenciais para que eu concluísse este curso.

Por fim, deixo meu sincero agradecimento ao meu orientador, professor Arthur, por sua orientação e dedicação durante todo o processo. Agradeço ainda a todos os professores que fizeram parte da minha formação, pois foram fundamentais para o meu aprendizado e crescimento acadêmico.

A todos vocês, o meu muito obrigado.

RESUMO

Modelos de linguagem de larga escala (LLMs) têm sido amplamente utilizados para automatizar tarefas de desenvolvimento de software. Embora apresentem bons resultados em diferentes linguagens e domínios, esses modelos ainda enfrentam dificuldades na geração de código paralelo, especialmente em contextos de computação de alto desempenho (HPC). Essa limitação decorre, em grande parte, da baixa presença de exemplos de programação paralela em datasets de pré-treinamento, incluindo APIs como CUDA, MPI e OpenMP, além da falta de informações sobre dependências de dados que emergem durante a paralelização.

Este trabalho propõe o refinamento (fine-tuning) de um Small Language Model (SLM) utilizando um dataset especializado exclusivamente em CUDA. O objetivo é avaliar o impacto da especialização na geração de código paralelo mais correto e funcional. Para isso, treinamos uma família de modelos refinados e os comparamos ao estado da arte, o HPC-Coder-V2, empregando o benchmark ParEval para medir métricas de compilação e execução. Os resultados mostram que modelos menores, quando especializados, podem alcançar desempenho competitivo ou superior a modelos maiores, destacando o potencial dos SLMs para aplicações em HPC.

Palavras-chave: Finetuning, CUDA, SLM, HPC.

ABSTRACT

Large Language Models (LLMs) have been increasingly adopted to automate and accelerate software development tasks. Despite their strong performance in a variety of programming languages and contexts, existing models still struggle to generate correct and efficient parallel code, particularly in high-performance computing (HPC) scenarios. This limitation arises mainly from the scarcity of parallel programming examples in pre-training datasets, especially those involving APIs such as CUDA, MPI, and OpenMP, as well as insufficient modeling of data dependencies and parallel execution patterns.

This work investigates the fine-tuning of a Small Language Model (SLM) using a domain-specialized dataset focused exclusively on CUDA. The goal is to evaluate the impact of specialization on the quality, correctness, and performance of generated parallel code. We train a family of refined models and compare them against the state-of-the-art HPC-Coder-V2, using the ParEval benchmark to assess build and execution correctness. Our results demonstrate that compact models, when trained with highly specialized datasets, can achieve competitive or superior performance relative to larger models, reinforcing the potential of domain-specific SLMs for HPC code generation.

Keywords: Finetuning, CUDA, HPC, SLM.

LISTA DE ABREVIATURAS E SIGLAS

LLM	Large Language Models
SLM	Small Language Models
HPC	High-Performance Computing

LISTA DE FIGURAS

Figura 4.1 Comparação das configurações CUDA 32

Figura 5.1 Progressão da média $build@k$ entre os modelos GPPD-HPC-CUDA-Coder e o modelo base 35
Figura 5.2 Resultados de $build@1$ por tipo de problema entre os modelos GPPD-HPC-CUDA-Coder e o modelo base 36
Figura 5.3 Progressão da média $pass@k$ entre os modelos GPPD-HPC-CUDA-Coder e o modelo base 37
Figura 5.4 Resultado de $pass@1$ por tipo de problema entre os modelos GPPD-HPC-CUDA-Coder e o modelo base 38
Figura 5.5 Progressão da média de $build@k$ entre os modelos refinados 39
Figura 5.6 Resultados de $build@1$ por tipo de problema entre os modelos refinados 40
Figura 5.7 Progressão da média de $pass@k$ entre todos modelos 41
Figura 5.8 Resultado de $pass@1$ por tipo de problema entre os modelos refinados 41

SUMÁRIO

1 INTRODUÇÃO 15
1.1 Objetivos 17
1.2 Estrutura do Texto 17
2 REVISÃO BIBLIOGRÁFICA 19
2.1 Programação Paralela em CUDA 19
2.2 Modelos de Linguagem 20
2.3 Benchmarks e Conjuntos de Dados para HPC 22
3 TRABALHOS RELACIONADOS 25
4 METODOLOGIA 29
4.1 Ambiente de execução 29
4.2 Refinamento da SLM 30
4.2.1 Coleta e Filtragem dos Datasets 30
4.2.2 Treinamento e Refinamento do Modelo 30
4.3 Realizando o benchmark 32
5 RESULTADOS 35
5.1 Análise entre os modelos refinados da família GPPD-HPC-CUDA-Coder 35
5.2 Comparação com o estado da arte: HPC-Coder-V2 37
5.3 Análise de eficiência computacional 39
6 CONCLUSÃO 43
REFERÊNCIAS 45

1 INTRODUÇÃO

Os modelos de linguagem de larga escala (LLMs, do inglês *Large Language Models*) tornaram-se amplamente populares nos últimos anos, consolidando-se como ferramentas indispensáveis no cotidiano de desenvolvedores de software. De acordo com a pesquisa anual promovida pelo GitHub, atualmente, 92% dos profissionais da área utilizam LLMs de forma recorrente em seu ambiente de trabalho (SHANI, 2024). Esses modelos, sejam de linguagem geral ou voltados especificamente para tarefas de programação, como o StarCoder 2 (LOZHKOV et al., 2024), são treinados em extensos conjuntos de dados (*datasets*) que abrangem uma variedade significativa de linguagens de programação, incluindo Java, Python, C, C++ e muitas outras. O resultado desse treinamento intensivo é a capacidade dos LLMs de desempenhar com eficiência diversas tarefas relacionadas ao desenvolvimento de software, tais como completamento automático, geração de novos trechos de código, refatoração e explicação de funções complexas.

A programação paralela consiste, em linhas gerais, na decomposição de um problema computacional em múltiplas partes que podem ser executadas simultaneamente, com o objetivo de reduzir o tempo de execução e explorar melhor os recursos de hardware disponíveis (CHANDY, 1989). Essa abordagem é central na *computação de alto desempenho* (HPC, do inglês *High Performance Computing*), na qual se fazem presentes arquiteturas especializadas, como unidades de processamento gráfico (GPUs, do inglês *Graphics Processing Units*), sistemas distribuídos e processadores com múltiplos núcleos. A correta implementação de algoritmos paralelos exige o domínio de conceitos como identificação e tratamento de dependências de dados, estratégias de sincronização entre tarefas e balanceamento de carga entre recursos computacionais. Além disso, o uso de bibliotecas e interfaces específicas, como *CUDA* (do inglês *Compute Unified Device Architecture*) para aceleração em GPUs, *MPI* (do inglês *Message Passing Interface*) para comunicação em sistemas distribuídos e *OpenMP* (do inglês *Open Multi-Processing*) para paralelismo em memória compartilhada, é fundamental para que o código resultante seja não apenas funcional, mas também eficiente e escalável. Por esses motivos, a escrita de código paralelo tende a ser substancialmente mais complexa do que o desenvolvimento de soluções sequenciais.

Diante dessa complexidade, é natural investigar se LLMs, já consolidados como assistentes de programação geral, podem também apoiar a geração de código paralelo. Trabalhos recentes têm explorado exatamente essa direção, avaliando LLMs na parale-

lização de aplicações e na geração de *kernels* para diferentes modelos de programação voltados a HPC. Por exemplo, Dias, Beck and Lorenzon (2024) empregam modelos da família GPT para produzir versões paralelas, com diretivas OpenMP, de aplicações baseadas em computação *stencil*, analisando tanto a correção funcional quanto o impacto no desempenho. Estudos dessa natureza sugerem que LLMs conseguem, em vários casos, gerar soluções paralelas executáveis e com ganhos de desempenho em relação às versões sequenciais, reforçando seu potencial como ferramenta de apoio ao desenvolvimento em ambientes de alto desempenho.

Apesar desses resultados promissores, a literatura também evidencia limitações importantes quando LLMs pré-treinados são confrontados com *prompts* que envolvem programação paralela e HPC em cenários mais gerais. Conforme apontado por Chaturvedi et al. (2024), um fator central é a escassez de exemplos relevantes nos *datasets* utilizados no treinamento desses modelos, que raramente incluem códigos que utilizam de forma sistemática APIs como OpenMP, MPI ou CUDA. Em consequência, os LLMs tendem a falhar na identificação das dependências de dados e na escolha de estratégias de paralelização adequadas, o que frequentemente resulta em soluções incorretas, ineficientes ou pouco escaláveis. Esses desafios indicam que, para explorar plenamente o potencial dos modelos de linguagem em HPC, é necessário adaptá-los ou especializá-los para esse domínio, seja por meio de *fine-tuning*, seja por meio de arquiteturas ou fluxos de uso específicos.

O refinamento de LLMs de grande porte, entretanto, envolve custos computacionais e financeiros elevados, uma vez que exige hardware robusto e grande capacidade de memória para acomodar centenas de bilhões de parâmetros. Esse cenário torna difícil manter modelos dessa escala atualizados e bem ajustados a domínios específicos, como o de programação paralela. Nesse contexto, ganha relevância a proposta dos *Small Language Models* (SLMs). Conforme discutido por Belcak et al. (2025), modelos menores podem ser treinados e implantados com custo significativamente reduzido e, ainda assim, oferecer desempenho competitivo em tarefas especializadas. Dessa forma, SLMs surgem como uma alternativa mais eficiente e sustentável para a construção de assistentes voltados à geração e ao aprimoramento de código paralelo, especialmente em aplicações agentivas e de ciclo iterativo, nas quais a interação frequente e a capacidade de adaptação ao contexto são fatores essenciais.

1.1 Objetivos

O objetivo geral deste trabalho é investigar o impacto da especialização de um SLM no contexto da geração de código paralelo em CUDA, por meio do processo de refinamento utilizando um *dataset* dedicado a essa tecnologia. A proposta central é avaliar se modelos de menor porte, quando treinados especificamente para um domínio restrito, podem alcançar desempenho competitivo em comparação a LLMs maiores e generalistas. Para alcançar esse objetivo geral, estabelecem-se os seguintes objetivos específicos:

- Construir o conjunto de dados utilizado no refinamento, automatizando o download dos datasets selecionados e filtrando apenas as amostras em CUDA para compor o *dataset* final.
- Realizar o processo de *fine-tuning* de um SLM utilizando o conjunto de dados especializado.
- Avaliar o desempenho do modelo refinado em tarefas de geração de código paralelo CUDA, utilizando o *benchmark* **ParEval** (NICHOLS et al., 2024a).
- Comparar os resultados obtidos pelo SLM refinado com o desempenho de LLMs generalistas disponíveis publicamente.

1.2 Estrutura do Texto

Além deste capítulo introdutório, este trabalho está organizado da seguinte forma.

O Capítulo 2 apresenta os fundamentos teóricos que sustentam o trabalho, cobrindo conceitos essenciais sobre Large Language Models (LLMs) e Small Language Models (SLMs), a arquitetura Transformer e seus mecanismos internos, além de princípios de programação paralela com CUDA. O capítulo também contextualiza o benchmark ParEval e o dataset HPC-INSTRUCT, descrevendo sua relevância para a avaliação e especialização de modelos de linguagem em cenários de computação de alto desempenho.

O Capítulo 3 discute pesquisas e modelos previamente desenvolvidos para geração de código paralelo, analisando suas abordagens, limitações e métricas de avaliação. A seção posiciona o presente estudo em relação ao estado da arte, evidenciando as lacunas existentes e justificando a necessidade de técnicas especializadas para geração automática de código eficiente em ambientes HPC.

O Capítulo 4 descreve detalhadamente as etapas de construção do dataset utili-

zado, o processo de refinamento do SLM escolhido, o ambiente experimental empregado nos testes e a estratégia adotada para coleta e análise das métricas de benchmark. Esse capítulo estabelece o fluxo metodológico que orienta toda a investigação, ressaltando as decisões técnicas que permitem avaliar a capacidade do modelo de gerar código paralelo de qualidade.

O Capítulo 5 apresenta os experimentos realizados e analisa comparativamente o desempenho do modelo refinado frente a LLMs de maior porte. A discussão aborda tanto métricas quantitativas quanto características qualitativas das soluções geradas, destacando ganhos, limitações e o impacto das otimizações aplicadas ao modelo menor.

Por fim, o Capítulo 6 sintetiza as contribuições alcançadas ao longo da pesquisa, refletindo sobre os avanços obtidos e os desafios ainda existentes para o uso de modelos de linguagem na geração de código paralelo. O capítulo também propõe direções futuras plausíveis, como aprimoramentos no processo de especialização, expansão do dataset e integração com ferramentas de análise de desempenho em HPC.

2 REVISÃO BIBLIOGRÁFICA

Neste capítulo, apresentamos os fundamentos teóricos que sustentam este trabalho, abrangendo desde os princípios de programação paralela em CUDA até os avanços recentes em modelos de linguagem baseados na arquitetura Transformer. São discutidos tanto os modelos de larga escala (LLMs) quanto os modelos compactos (SLMs), destacando suas características, limitações e aplicações no contexto de geração de código. Além disso, são apresentados os principais benchmarks e conjuntos de dados utilizados para avaliação e treinamento de modelos voltados à produção de código paralelo em ambientes de *High-Performance Computing* (HPC), com ênfase no *ParEval* e no *HPC-INSTRUCT*. Esses elementos fornecem a base conceitual necessária para compreender a motivação, os desafios e as contribuições técnicas desta pesquisa.

2.1 Programação Paralela em CUDA

O CUDA é um modelo de programação paralela desenvolvido pela NVIDIA NVIDIA Corporation (2025), que permite explorar o paralelismo massivo das GPUs por meio da execução de *kernels* compostos por milhares de *threads*. Essas threads são organizadas na hierarquia **grid** $\rightarrow$ **blocks** $\rightarrow$ **threads**, onde cada bloco é executado em um *Streaming Multiprocessor* (SM). Dentro do SM, as threads são agrupadas em *warps* de 32 threads e executam segundo o paradigma **SIMT** (Single-Instruction, Multiple-Threads), o que viabiliza alta taxa de paralelismo e eficiente ocultação de latência.

O desempenho de kernels CUDA depende fortemente do uso correto da hierarquia de memória da GPU. A memória global possui alta latência e deve ser acessada de forma **contígua** por threads de um mesmo warp, enquanto a memória compartilhada, localizada dentro do SM, oferece baixa latência e é fundamental para minimizar acessos à memória global. Registradores representam a forma mais rápida de armazenamento para variáveis locais, mas seu uso excessivo pode reduzir a ocupação do SM. Já memórias constantes e de textura fornecem acessos otimizados para padrões específicos de leitura.

A coordenação entre threads ocorre principalmente por meio da função *__syncthreads()*, que sincroniza todas as threads de um bloco, e por operações atômicas, que permitem manipulação concorrente de valores compartilhados. Como não há sincronização direta entre blocos durante a execução de um kernel, estruturas mais complexas exigem múltiplos lançamentos de kernel ou o uso de *cooperative groups*.

A arquitetura CUDA também permite sobreposição entre computação e movimentação de dados por meio de *streams*, que executam operações de forma independente. Transferências assíncronas (*cudaMemcpyAsync*) e execução concorrente de vários kernels possibilitam melhor aproveitamento dos recursos da GPU, reduzindo o custo de comunicação entre CPU e GPU, um dos principais gargalos em aplicações paralelas.

2.2 Modelos de Linguagem

Os modelos de linguagem modernos têm desempenhado um papel central em diversas tarefas de processamento de linguagem natural, geração de código, análise textual e sistemas multimodais. A evolução desses modelos está diretamente atrelada ao surgimento da arquitetura *Transformer*, introduzida por (VASWANI et al., 2017), que redefiniu o paradigma de modelagem de sequências ao substituir completamente mecanismos recorrentes e convolucionais pelo uso exclusivo de atenção. Essa mudança possibilitou ganhos expressivos em paralelização, estabilidade numérica e desempenho em tarefas como tradução automática, tornando-se a base dos *Large Language Models* (LLMs).

A arquitetura do Transformer fundamenta-se no mecanismo de atenção por produto escalar escalonado (*Scaled Dot-Product Attention*), definido por:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

em que Q, K e V representam as matrizes de *queries*, *keys* e *values*. Para capturar múltiplos padrões contextuais simultaneamente, o Transformer utiliza atenção com múltiplas cabeças (*Multi-Head Attention*):

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O,$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V).$$

Como o Transformer não possui recorrência, informações de posição são incorporadas via codificações posicionais (*Positional Encodings*), definidas por:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right),$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right).$$

A combinação desses componentes, associada a blocos empilhados de codificação e decodificação, conexões residuais e normalização de camada, tornou o Transformer uma arquitetura altamente paralelizável. Isso permitiu o treinamento de LLMs com centenas de bilhões de parâmetros, como GPT (ACHIAM et al., 2023) e Codex (CHEN et al., 2021), capazes de gerar texto e código com alto grau de coerência. Conforme demonstrado por (VASWANI et al., 2017), o Transformer também supera arquiteturas recorrentes e convolucionais em diversos benchmarks, estabelecendo novos recordes de BLEU score e reduzindo significativamente o custo computacional requerido.

Entretanto, apesar de sua expressividade, LLMs dessa escala apresentam limitações práticas importantes. O custo computacional para executar ou ajustar modelos com dezenas ou centenas de bilhões de parâmetros é elevado, exigindo hardwares especializados, grande capacidade de memória e alto consumo energético. Essas restrições dificultam tanto o uso local quanto o *fine-tuning* para aplicações específicas, especialmente em ambientes com recursos limitados.

Nesse cenário, cresce o interesse por modelos menores, conhecidos como *Small Language Models* (SLMs). Segundo Belcak et al. (2025), modelos com até 10 bilhões de parâmetros podem ser considerados SLMs. Quando bem pré-treinados e ajustados com técnicas adequadas de instrução e prompting, esses modelos alcançam desempenho competitivo em diversas tarefas, incluindo raciocínio, análise semântica e geração de código.

Os SLMs oferecem vantagens significativas em relação aos LLMs tradicionais: custo computacional reduzido, menor latência na inferência, menor pegada energética e maior viabilidade de operação em dispositivos locais, sistemas embarcados e ambientes *on-edge*. Além disso, segundo Belcak et al. (2025), tendem a ser mais transparentes, interpretáveis e fáceis de adaptar, tornando-se especialmente atraentes para aplicações que exigem controle, auditabilidade e segurança.

Assim, enquanto os LLMs representam o estado da arte em desempenho bruto e capacidade de generalização, os SLMs surgem como uma alternativa prática, econômica e eficiente para cenários especializados, proporcionando um equilíbrio entre capacidade computacional e acessibilidade.

2.3 Benchmarks e Conjuntos de Dados para HPC

A geração automática de código paralelo é um dos desafios mais complexos para modelos de linguagem, devido à diversidade de modelos de execução, estruturas de dados e requisitos de desempenho presentes em aplicações de *High-Performance Computing* (HPC). Para permitir o estudo sistemático dessas capacidades, a literatura recente apresenta dois recursos fundamentais: o *ParEval*, voltado à avaliação rigorosa de LLMs, e o *HPC-INSTRUCT*, destinado ao treinamento e aperfeiçoamento de modelos especializados em código paralelo.

O *ParEval* é um *benchmark* abrangente proposto por Nichols et al. (2024a) para medir a capacidade de LLMs na geração de código paralelo em diferentes paradigmas. Ele contém 420 tarefas computacionais de 12 classes distintas, incluindo álgebra linear densa e esparsa, ordenação, FFTs e algoritmos de grafos, cada uma implementada em sete modelos de programação: sequencial, OpenMP, MPI, MPI+OpenMP, Kokkos, CUDA e HIP. Ao total, o conjunto resulta em 2.940 *prompts* cuidadosamente elaborados para testar a correção, a aderência às APIs de paralelização e o desempenho do código gerado.

Além da métrica tradicional de *pass@k*, o *ParEval* introduz medidas específicas para HPC: $\textit{speedup}_n@k$ e $\textit{efficiency}_n@k$, que avaliam respectivamente o ganho de desempenho obtido e a eficiência do uso dos recursos paralelos. Essas métricas permitem analisar não apenas a correção funcional do código, mas sua adequação às demandas de escalabilidade e desempenho presentes em ambientes reais de HPC.

Enquanto o *ParEval* foca na avaliação, o *HPC-INSTRUCT* (CHATURVEDI et al., 2024) concentra-se na criação de dados para o treinamento de LLMs especializados. O conjunto é composto por mais de 122 mil amostras sintéticas de *instrução-resposta*, geradas a partir de cerca de 125 mil trechos de código paralelo reais extraídos de repositórios públicos. Esses trechos cobrem modelos como OpenMP, MPI, CUDA e HIP, e utilizam linguagens amplamente adotadas em HPC, como C, C++ e Fortran.

As instruções foram produzidas por modelos avançados, como Gemini-Pro, DBRX, Llama-3-70B e Mixtral-8x7B, seguindo um processo sistemático que envolve *seed snippets* do *The Stack v2* (LOZHKOV et al., 2024). O dataset contempla desde problemas clássicos de paralelização até tradução entre paradigmas (por exemplo, converter código CUDA para OpenMP), otimização de implementações e geração de versões paralelas a partir de código sequencial.

O *HPC-INSTRUCT* foi utilizado para o refinamento de modelos como *DeepSeek-*

Coder (GUO et al., 2024) e *DeepSeek-Coder-V2* (ZHU et al., 2024), resultando no *HPC-Coder-V2*, que alcançou desempenho superior entre modelos abertos para geração de código paralelo. Estudos apresentados em Chaturvedi et al. (2024) mostram que tanto a qualidade quanto a quantidade dos dados influenciam significativamente o desempenho, e que técnicas como *instruction masking* podem aumentar a robustez do modelo.

Assim, *ParEval* e *HPC-INSTRUCT* desempenham papéis complementares: o primeiro estabelece um protocolo padronizado de avaliação para entender limites e capacidades atuais das LLMs em HPC, enquanto o segundo fornece uma base escalável e diversificada de dados sintéticos de alta qualidade para treinar e aperfeiçoar modelos especializados nesse domínio. Em conjunto, esses recursos constituem a base metodológica necessária para avançar o estado da arte em geração automática de código paralelo.

3 TRABALHOS RELACIONADOS

Neste capítulo, apresentamos uma revisão abrangente dos principais trabalhos desenvolvidos sobre o uso de LLMs na geração, modernização e otimização de código em ambientes de HPC. Inicialmente, discutimos pesquisas que exploram diretamente a aplicação de modelos generativos em tarefas típicas de programação paralela. Em seguida, analisamos os esforços que avançam em direção à criação de modelos especializados por meio de *fine-tuning*, destacando suas contribuições técnicas e limitações. A partir dessa análise, evidenciamos como a área tem evoluído e quais lacunas permanecem abertas, motivando diretamente o desenvolvimento deste trabalho.

O artigo de Dias, Beck and Lorenzon (2024) apresenta um estudo sobre a aplicação de modelos da família GPT na geração automática de código paralelo em aplicações de Computação Stencil. Os autores exploraram o uso dos modelos GPT-3.5 Turbo e GPT-4o para gerar código paralelo em oito aplicações com diferentes dimensões e estruturas de dados, executadas em três arquiteturas multicore. Os resultados demonstraram que ambos os modelos foram capazes de produzir códigos paralelos que superaram o desempenho das versões sequenciais, embora o GPT-3.5 Turbo tenha alcançado desempenho médio 15% superior ao GPT-4o. Apesar disso, ainda foram observadas limitações na identificação de dependências de dados e na geração de código eficiente em cenários complexos.

Outros trabalhos recentes evidenciam o avanço da utilização de LLMs generalistas em tarefas de HPC. O *ChatBLAS*, apresentado por Lei et al. (2023), demonstra a capacidade de modelos generativos na criação automática de rotinas BLAS otimizadas e portáveis para diferentes arquiteturas CPU/GPU. Os autores mostram que LLMs podem produzir implementações eficientes, capazes de competir com bibliotecas otimizadas manualmente.

Igualmente relevante é o *ChatHPC*, proposto por Lara et al. (2025), que apresenta um ecossistema baseado em LLMs para apoiar o desenvolvimento de software científico. O trabalho demonstra que modelos relativamente pequenos podem auxiliar tarefas como paralelização, otimização, portabilidade de código e instrumentação, obtendo em determinados casos níveis de confiabilidade superiores aos de LLMs muito maiores.

Além da geração de código, LLMs também têm sido explorados para automatizar etapas de validação e verificação em ambientes HPC. O estudo Munley, Jarmusch and Chandrasekaran (2024) investiga o uso de modelos como DeepSeek-Coder e GPT-4-Turbo na geração automática de testes para validação de compiladores OpenACC. Os

autores mostram que milhares de testes funcionais podem ser gerados automaticamente, cobrindo cenários complexos de uso de diretivas paralelas e reduzindo significativamente o trabalho manual.

Outro exemplo é o estudo Zhou, Hawkins and Gentine (2024), que demonstra a tradução semi-automática de componentes de modelos climáticos escritos em Fortran para versões modernas em Python/JAX. O trabalho mostra que a tradução assistida por LLMs possibilita acelerações significativas no desempenho, especialmente quando executada em GPU, além de habilitar a diferenciação automática, um recurso inexistente na implementação original.

Diversos trabalhos também se dedicam à utilização de LLMs generalistas para programação, como o Codex (CHEN et al., 2021) e o Code Llama (ROZIERE et al., 2023). Embora poderosos, tais modelos não são originalmente voltados para desafios específicos do ambiente HPC, o que motivou o surgimento de abordagens especializadas através de fine-tuning.

Nesse contexto, o *HPC-Coder*, proposto por Nichols et al. (2024b), representa um dos primeiros esforços estruturados para criar um LLM especializado em HPC. O modelo foi ajustado com um amplo conjunto de códigos científicos nas linguagens C e C++. Além disso, os autores introduziram benchmarks específicos para geração de código paralelo, anotação automática de laços com OpenMP e predição de impacto no desempenho. Os resultados mostram acurácia de até 97% na inserção correta de diretivas OpenMP e até 92% de precisão na previsão de mudanças de desempenho.

Dando continuidade a essa linha, o *HPC-Coder-V2*, apresentado por Chaturvedi et al. (2024), evolui a proposta anterior ao incorporar linguagens e modelos de execução de baixo nível, incluindo Fortran, CUDA, MPI, OpenCL e OpenMP. O diferencial do modelo é o uso do dataset sintético *HPC-INSTRUCT*, composto por mais de 122 mil pares instrução-resposta voltados para paralelização e otimização.

Além destes, outros modelos especializados surgem com foco ainda mais restrito. O *OMPGpt*, de Chen et al. (2024), dedica-se exclusivamente à recomendação de diretivas OpenMP, aprendendo padrões de paralelismo e dependências de dados a partir de código anotado. Os resultados mostram precisão elevada e competitividade na paralelização automática de laços complexos.

Seguindo a mesma lógica de especialização, o trabalho de Kadosh et al. (2024) apresenta o *MonoCoder*, modelo treinado exclusivamente em código HPC nas linguagens C e C++. O modelo demonstra desempenho superior a LLMs muito maiores em

tarefas de completude e geração de paralelismo, reforçando a hipótese de que modelos menores, quando altamente especializados, podem capturar características essenciais do código científico.

Apesar dos avanços apresentados pelos modelos especializados discutidos nesta seção, observa-se que a maior parte das soluções existentes concentra-se em LLMs de médio ou grande porte, com foco predominante em linguagens tradicionais de HPC como C, C++ e OpenMP. Poucos trabalhos investigam de forma sistemática o impacto da especialização em modelos de menor escala, especialmente para a geração de código CUDA.

Nesse sentido, este trabalho diferencia-se ao investigar o refinamento de um *Small Language Model* dedicado exclusivamente à programação paralela em GPUs, utilizando um *dataset* especializado em CUDA. Tal abordagem busca avaliar a viabilidade de modelos compactos, mais leves e economicamente treináveis, competirem com LLMs maiores em tarefas de geração de código paralelo.

4 METODOLOGIA

Neste capítulo, descrevemos de forma detalhada os procedimentos adotados para preparar o ambiente de execução, conduzir o refinamento do modelo e a avaliação experimental por meio do benchmark *ParEval*. São apresentadas as etapas de configuração da infraestrutura computacional, construção dos datasets, processo de *fine-tuning* supervisionado e, por fim, a execução sistemática dos testes utilizados para mensurar o desempenho dos modelos. Essa metodologia estabelece as bases necessárias para garantir reprodutibilidade, rigor técnico e a consistência na análise dos resultados.

4.1 Ambiente de execução

Os experimentos de refinamento e execução do benchmark foram realizados utilizando a infraestrutura de computação de alto desempenho do *Grupo de Processamento Paralelo e Distribuído* (GPPD) da Universidade Federal do Rio Grande do Sul (UFRGS), disponível em <https://gppd-hpc.inf.ufrgs.br>. Inicialmente, o acesso ao ambiente de computação foi realizado de forma remota por meio do protocolo *SSH*, conectando-se à máquina de acesso (portal) do GPPD. Em seguida, foram clonados os repositórios necessários a partir do *GitHub*, sendo eles o *GPPD-HPC-Coder* (<https://github.com/gdsantana/GPPD-HPC-Coder>) e o *ParEval* (<https://github.com/gdsantana/ParEval>).

Para o gerenciamento das dependências e maior controle sobre o ambiente de execução, foi criado e configurado um ambiente virtual *Python* (`venv`) contendo todas as bibliotecas requeridas para as etapas de refinamento do modelo e execução dos experimentos. Após a preparação do ambiente, procedeu-se à alocação de recursos computacionais na partição *tupi*, por meio do comando `salloc`, e ao acesso remoto à referida partição via *SSH*. A escolha da partição *tupi* deve-se às suas máquinas possuírem uma GPU *NVIDIA GeForce RTX 4090* contendo 24 GB de memória, que será essencial para a execução dos scripts.

Por fim, com o ambiente devidamente configurado e os recursos computacionais alocados, os repositórios correspondentes, seja o destinado ao refinamento do modelo ou o referente ao benchmark, foram acessados remotamente. A partir deles, executaram-se os comandos e scripts descritos nas seções específicas, conduzindo as etapas de treinamento e avaliação de forma controlada dentro da infraestrutura do GPPD-HPC.

4.2 Refinamento da SLM

O processo de refinamento da SLM foi conduzido em duas etapas principais: (i) preparação e filtragem dos dados de treinamento e (ii) treinamento supervisionado para especialização em código *CUDA*.

4.2.1 Coleta e Filtragem dos Datasets

Inicialmente, foi desenvolvido um arquivo em Python, denominado *dataset-loader.py*, responsável por realizar o download automático de três conjuntos de dados de instrução disponíveis na plataforma *Hugging Face*: *HPC-INSTRUCT*, *Evol-INSTRUCT-Code* (LUO et al., 2023) e *MagicaCoder-OSS* (WEI et al., 2023). Os datasets *Evol-INSTUCT-Code* e *MagicaCoder-OSS* são conjuntos de dados sintéticos e semi-sintéticos de instrução de código que ,embora não sejam específicos para HPC, esses dois conjuntos foram incluídos no treinamento para melhorar a capacidade de geração de código do modelo, como apresentado por Chaturvedi et al. (2024).

Após o download, é aplicado um filtro seletivo sobre o campo *Language* do dataset *HPC-INSTRUCT*, de modo a reter apenas as amostras referentes à linguagem *CUDA*. Essa filtragem assegura que o modelo seja exposto a exemplos consistentes de programação paralela voltada a *GPUs*, maximizando sua especialização no domínio de computação de alto desempenho (HPC).

4.2.2 Treinamento e Refinamento do Modelo

A segunda fase corresponde ao refinamento propriamente dito, em que ocorre o treinamento supervisionado do modelo base. Para essa etapa, foi desenvolvido um script em Python que realiza o carregamento do modelo-base a partir da plataforma *Hugging Face*. A escolha do *DeepSeek-Coder* (GUO et al., 2024) como modelo base deve-se à sua compatibilidade arquitetural e metodológica com o modelo utilizado por Chaturvedi et al. (2024) para o refinamento do HPC-Coder-V2, o que reduz variáveis de comparação e permite uma análise mais direta do impacto do refinamento proposto.

No processo de refinamento, foi utilizado o script de carregamento dos conjuntos de dados descritos na Seção 4.2.1. O *fine-tuning* supervisionado foi conduzido sobre esse

conjunto de dados, aplicando técnicas de *instruction masking* para preservar a capacidade de generalização do modelo e evitar o aprendizado de instruções incorretas eventualmente presentes nos dados de treinamento, conforme as recomendações de Chaturvedi et al. (2024). Essa etapa ajusta os pesos do modelo para otimizar sua capacidade de gerar respostas e códigos CUDA corretos, coerentes e computacionalmente eficientes.

A implementação do processo de refinamento foi desenvolvida utilizando o ecossistema *Transformers* da Hugging Face, empregando as bibliotecas `transformers` para carregamento e manipulação dos modelos de linguagem, `datasets` para processamento eficiente dos conjuntos de dados, e `peft` (*Parameter-Efficient Fine-Tuning*) para implementação da técnica LoRA (*Low-Rank Adaptation*). O treinamento supervisionado foi conduzido através da biblioteca `trl` (*Transformer Reinforcement Learning*), especificamente utilizando a classe `SFTTrainer` (*Supervised Fine-tuning Trainer*), que oferece otimizações específicas para o ajuste fino supervisionado de modelos de linguagem de grande escala.

Para otimização de memória e eficiência computacional, foi implementada quantização de 8 bits através da biblioteca `bitsandbytes`, permitindo o treinamento em hardware com recursos limitados. O carregamento e processamento dos dados foram realizados através de um módulo personalizado *dataset-loader.py*, utilizando processamento paralelo para formatação dos exemplos de treinamento. A configuração LoRA empregou rank $r = 16$, parâmetro alpha $\alpha = 32$ e dropout de 0.05, balanceando eficiência de parâmetros e capacidade de adaptação do modelo.

A execução prática do refinamento foi conduzida por meio do comando apresentado no Listagem 4.1, no qual são definidos o modelo base a ser refinado, o diretório destinado ao armazenamento dos *checkpoints* intermediários, o diretório de saída do modelo final refinado, o arquivo de *logs* para monitoramento do processo e as *flags* que habilitam o uso de *datasets* complementares ao conjunto principal de dados.

O processo de refinamento resultou em dois modelos: o primeiro, denominado *GPPD-HPC-CUDA-Coder-1.3b-full-dataset*, foi treinado utilizando todos os *datasets*, incluindo os adicionais; o segundo, denominado *GPPD-HPC-CUDA-Coder-1.3b-hpc-only-dataset*, empregou exclusivamente o conjunto de dados específico de HPC. Essa etapa teve como objetivo avaliar o impacto dos *datasets* de instruções gerais no aprimoramento da capacidade do modelo em gerar código CUDA. Para o modelo que utilizou todos os *datasets*, o processo completo de refinamento supervisionado demandou aproximadamente 13 horas, processando um total de 167.756 instruções.

```
python3 finetune_with_args.py \
  --model_name deepseek-ai/deepseek-coder-1.3b-base \
  --output_dir ./checkpoints/gppd-hpc-cuda-coder-1.3b-base \
  --save_dir ./models/gppd-hpc-cuda-coder-1.3b-base \
  --log_file ./logs/gppd-hpc-cuda-coder-1.3b-base.log \
  --use_evol_instruct \
  --use_magicoder
```

Listagem 4.1: Execução do script `finetune_with_args.py` para refinamento do modelo.

O processo completo de refinamento supervisionado do modelo demandou aproximadamente 13 horas, processando um conjunto de 167.756.

4.3 Realizando o benchmark

Inicialmente, foi necessário realizar ajuste em um arquivos de configuração do benchmark. No arquivo *build-configs.json*, modificou-se o caminho associado às configurações da plataforma *CUDA*, conforme ilustrado na Figura 4.1.

Em seguida, procedeu-se à compilação das dependências do projeto, seguindo o passo a passo descrito no arquivo *README.md* disponível no repositório.

Antes

```
        "cuda": {
        "CXX": "nvcc",
        "CXXFLAGS": "-std=c++17 ..."
        }
```

Depois

```
      "cuda": {
        "CXX": "/usr/local/cuda/bin/nvcc",
        "CXXFLAGS": "-std=c++17 ..."
      }
```

Figura 4.1: Comparação das configurações CUDA

Para a execução do benchmark *ParEval* foi aplicado um filtro ao arquivo *generation-prompts.json* com o objetivo de selecionar apenas os *prompts* relacionados à linguagem CUDA, de modo a reduzir o tempo de execução e concentrar a avaliação na geração de código paralelo. O resultado desse processo foi um novo arquivo contendo exclusivamente os *prompts* de CUDA, denominado de *cuda-only-prompts.json*, que serviu como entrada

para a etapas subsequente do benchmark.

Em seguida, foi executado o script *generate.py*, especificando o modelo a ser utilizado, podendo este ser carregado diretamente da plataforma *Hugging Face* ou a partir de um diretório local, juntamente com o arquivo filtrado contendo os *prompts* de CUDA, como ilustrado na listagem 4.2. Por padrão, o script gera 50 saídas (*outputs*) para cada *prompt* de entrada, parâmetro essencial para o cálculo posterior das métricas *pass@k*, *build@k*, entre outras.

```
python3 generate/generate.py \
--prompts prompts/cuda-only-prompts.json \
--model [caminho_modelo] \
--output [caminho_para_salvar_resultado].json \
--cache cache/[model_name].jsonl \
--do_sample \
--prompted
```

Listagem 4.2: Execução do script `generate.py` com parâmetros dinâmicos.

Durante os testes com modelos de maior porte, como aqueles contendo 6,7 bilhões de parâmetros, foi necessário ajustar o parâmetro *batch_size* para 4, reduzindo o grau de paralelismo da inferência a fim de possibilitar a execução dentro das limitações do hardware disponível. Portanto, o comando executado para modelos gerar os outputs para modelos de 6.7 bilhões de parâmetros foi o ilustrado na listagem 4.3:

```
python3 generate/generate.py \
--prompts prompts/cuda-only-prompts.json \
--model [caminho_modelo] \
--output [caminho_para_salvar_resultado].json \
--cache cache/[model_name].jsonl \
--batch_size 4 \
--do_sample \
--prompted
```

Listagem 4.3: Execução do script *generate.py* com parâmetros dinâmicos com *batch_size* alterado.

Após a geração das saídas, foi utilizado outro script da Listagem 4.4, denominado *run-all.py*, integrante do benchmark para validar cada resultado, verificando se o código gerado é capaz de ser compilado, executado e apresentar melhoria de desempenho em

relação às versões de referência. Com base nesses resultados, foram calculadas as principais métricas de avaliação do *ParEval*, incluindo *pass@k*, *build@k* e indicadores de performance.

```
python3 drivers/run-all.py \
[caminho_do_resultado_de_generate.py] \
-o [caminho_para_salvar_resultado] \
--include-models cuda \
--launch-configs drivers/launch-configs.json \
--build-configs drivers/build-configs.json \
--problem-sizes drivers/problem-sizes.json \
--yes-to-all
```

Listagem 4.4: Execução do script *run-all.py*.

O proximo passo foi criar um *data-frame* com o script *create-dataframe.py* utilizando como entrada para ele, o resultado do *run-all.py*, com o comando exemplificado em 4.5. E, em seguida, o dataframe recem gerado, é utilizado para o script *metrics.py*, no qual é executado a partidor do comando exemplificado na Listagem 4.6, para enfim gerar as métricas que serão utilizadas para comparar os resultados.

```
python3 analysis/create-dataframe.py \
[caminho_do_resultado_de_run_all.py] \
--output [caminho_para_salvar_data_frame]
```

Listagem 4.5: Execução do script *create-dataframe.py*.

```
python3 analysis/metrics.py \
[caminho_do_dataframe] \
--output [caminho_para_salvar_as_metricas] \
--problem-sizes drivers/problem-sizes.json \
--model-name [nome_do_modelo]
```

Listagem 4.6: Execução do script *metrics.py*.

O procedimento descrito foi aplicado de maneira uniforme a todos os modelos avaliados, variando apenas o valor do parâmetro *batch_size*, que foi ajustado para valores menores nos modelos de maior dimensão a fim de manter a viabilidade computacional dos experimentos.

5 RESULTADOS

Com base nas métricas de desempenho geradas pelo *benchmark ParEval* (NICHOLS et al., 2024a), foi possível realizar uma análise comparativa detalhada entre os modelos avaliados. O *ParEval* é amplamente reconhecido como o principal conjunto de testes para avaliação da capacidade de modelos de linguagem em gerar código paralelo, fornecendo métricas como $build@1$ e $pass@1$, que medem, respectivamente, a taxa de compilação bem-sucedida e a taxa de execução correta do código gerado.

5.1 Análise entre os modelos refinados da família GPPD-HPC-CUDA-Coder

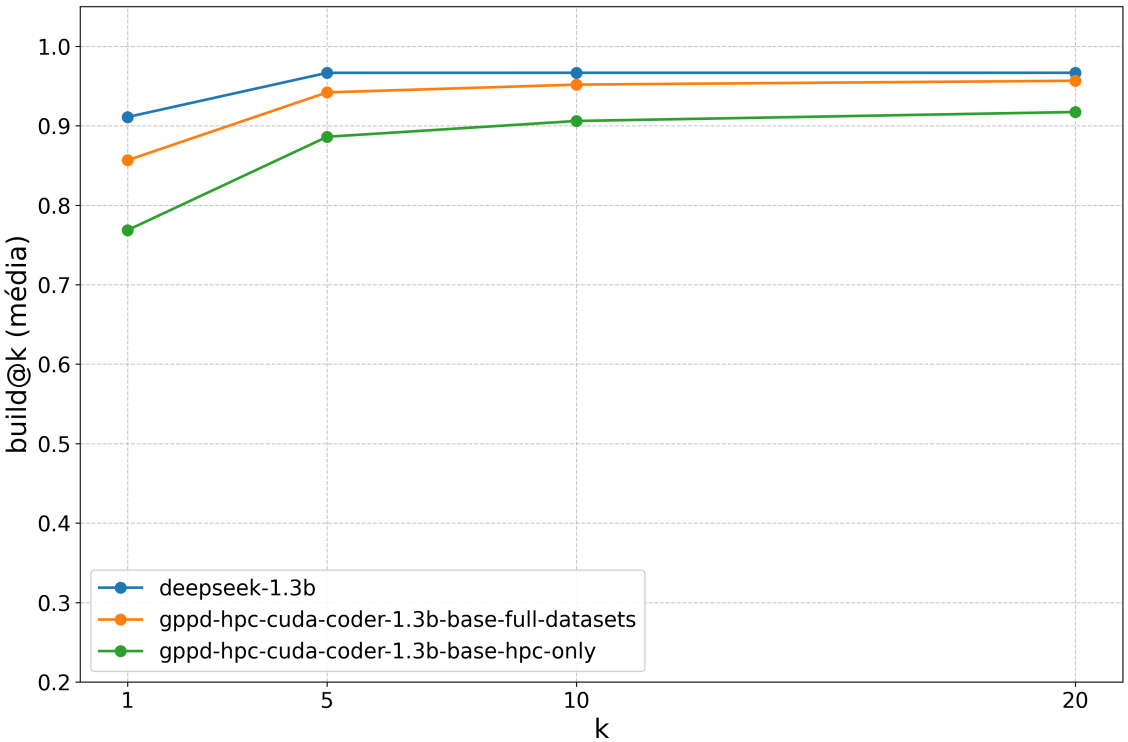


Figura 5.1: Progressão da média $build@k$ entre os modelos GPPD-HPC-CUDA-Coder e o modelo base

A análise inicial concentrou-se na comparação interna entre os modelos refinados da família *GPPD-HPC-CUDA-Coder*, com o objetivo de identificar aquele que apresenta o melhor desempenho geral ao gerar código CUDA. A Figura 5.1 ilustra o comportamento da média da métrica $build@k$ para os dois modelos da família GPPD frente ao modelo base *Deepseek-Coder-1.3b*. Observa-se uma consistência geral entre os três modelos, embora o modelo base apresente, em média, maior capacidade de gerar código compilável.

Essa tendência se torna ainda mais evidente quando analisamos a métrica $build@1$, que avalia a taxa de sucesso na primeira tentativa. Conforme apresentado na Figura 5.2, o modelo *Deepseek-Coder* supera os modelos refinados principalmente nas categorias *geometry*, *scan* e *sparse_la*, sugerindo que, apesar de não ser especializado, mantém robustez estrutural para compor código sintaticamente correto em diferentes domínios.

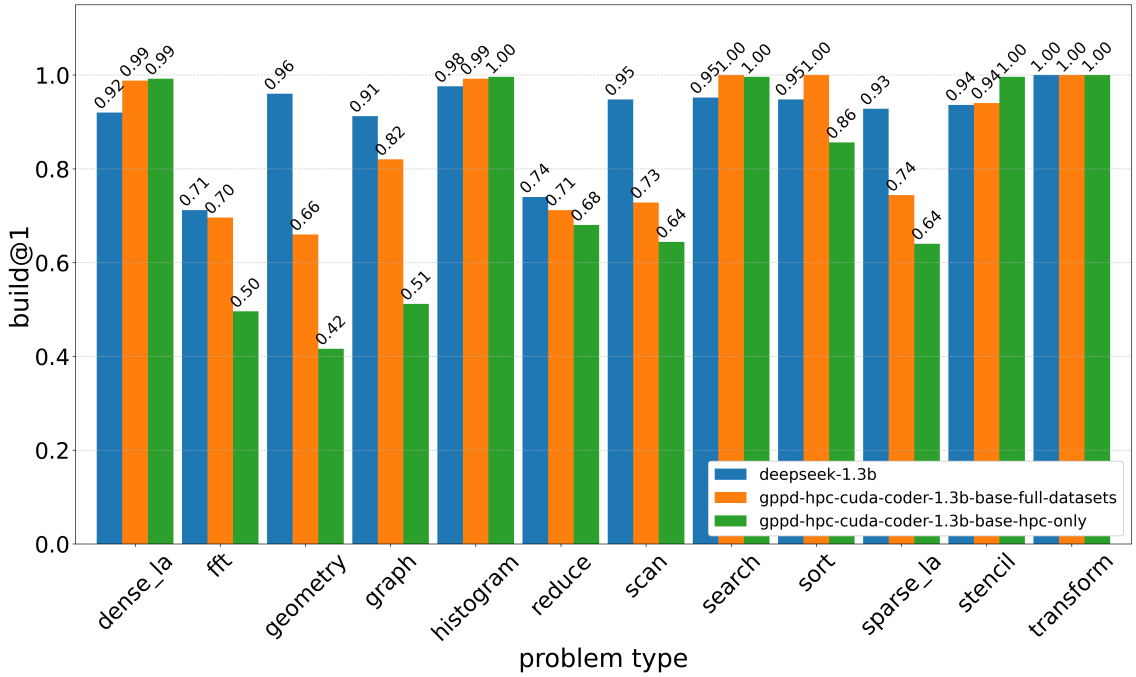


Figura 5.2: Resultados de $build@1$ por tipo de problema entre os modelos GPPD-HPC-CUDA-Coder e o modelo base

No entanto, a geração de código compilável representa apenas parte do problema, é essencial que o código produzido seja funcionalmente correto. Nesse sentido, a Figura 5.3 apresenta os resultados médios da métrica $pass@k$, evidenciando com maior clareza o impacto positivo do refinamento voltado à geração de código CUDA. Diferentemente do comportamento observado em $build@k$, os modelos da família *GPPD-HPC-CUDA-Coder* passam a demonstrar vantagem sobre o modelo base, produzindo código que além de compilar, também apresenta correção funcional. Entre eles, o modelo *GPPD-HPC-CUDA-Coder-full-dataset* destaca-se ao alcançar melhor média para $k \leq 10$, indicando maior estabilidade ao longo de múltiplas tentativas.

Ao aprofundarmos a análise para o caso mais restrito, considerando apenas $k = 1$, o cenário revela particularidades adicionais. Como ilustrado na Figura 5.4, o modelo *gppd-hpc-cuda-coder-hpc-only* apresenta desempenho superior em problemas da categoria *histogram*, enquanto o modelo *gppd-hpc-cuda-coder-full-dataset* se sobressai em tarefas de *stencil*. Esses resultados reforçam que, apesar das melhorias obtidas pelos modelos

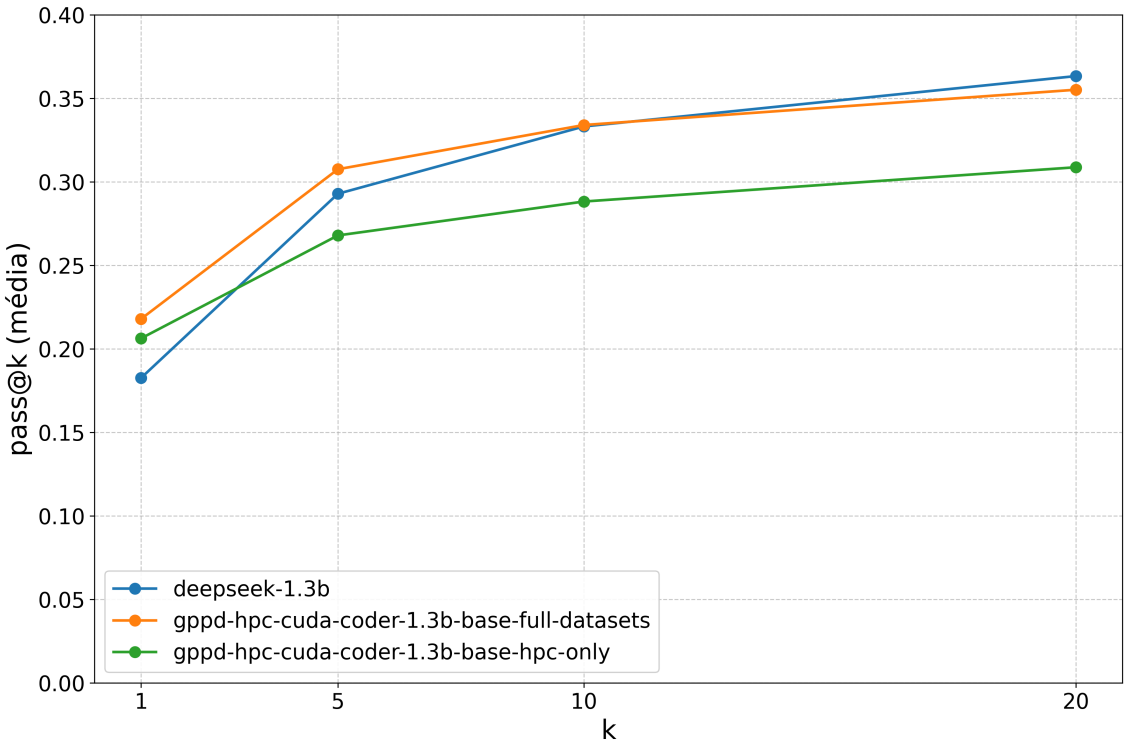


Figura 5.3: Progressão da média $pass@k$ entre os modelos GPPD-HPC-CUDA-Coder e o modelo base

refinados, a incorporação de *datasets* adicionais não específicos de HPC favorece apenas alguns tipos de problemas, indicando que seu impacto é seletivo e não generalizado.

5.2 Comparação com o estado da arte: HPC-Coder-V2

Após a análise comparativa entre os modelos da família *GPPD-HPC-CUDA-Coder-1.3b* e seu modelo base, *Deepseek-Coder-1.3b*, avançamos para a avaliação frente aos modelos de estado da arte em geração de código paralelo: os modelos da família *HPC-Coder-V2*. Para essa comparação, foram considerados tanto o modelo de mesma escala, *HPC-Coder-V2-1.3b*, quanto o modelo maior, *HPC-Coder-V2-6.7b*, com 1.3 bilhões e 6.7 bilhões de parâmetros, respectivamente. A Figura 5.5 apresenta o desempenho médio da métrica $build@k$ para os quatro modelos. Observa-se que a especialização dos modelos *GPPD* exclusivamente em código CUDA impacta positivamente sua capacidade de gerar código compilável, resultando em médias superiores às obtidas pelos modelos *HPC-Coder-V2*.

Essa diferença torna-se ainda mais evidente quando analisamos o caso restrito de $k = 1$, ilustrado na Figura 5.6. Nessa configuração, os modelos *GPPD-HPC-CUDA-*

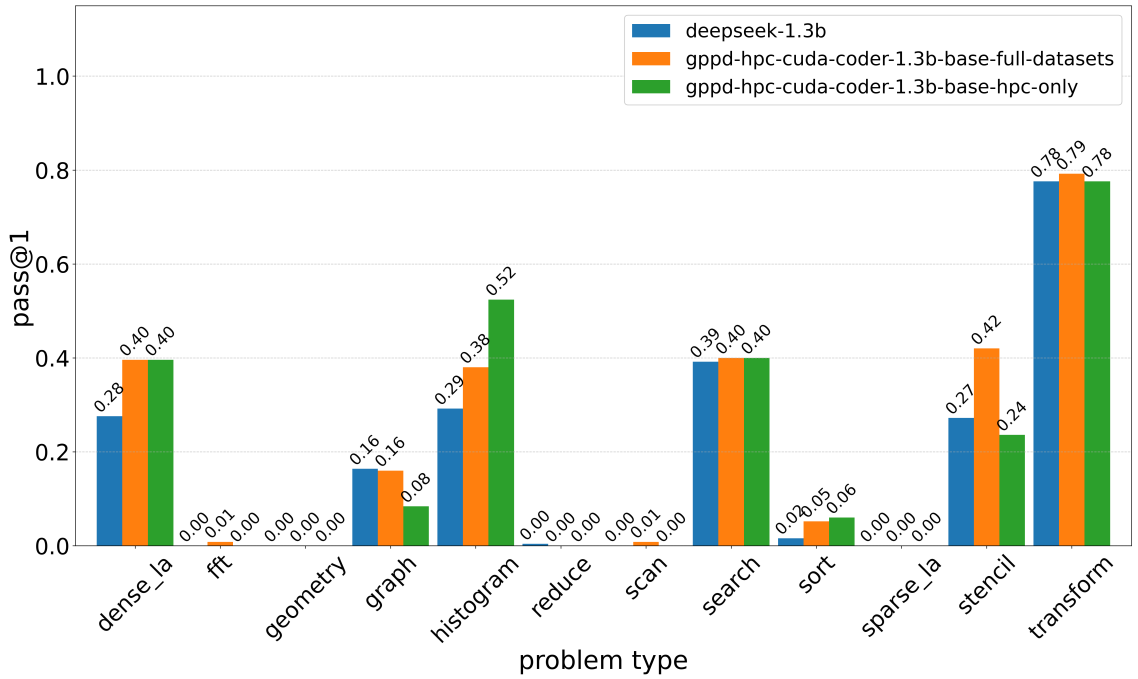


Figura 5.4: Resultado de $pass@1$ por tipo de problema entre os modelos GPPD-HPC-CUDA-Coder e o modelo base

Coder apresentam desempenho equivalente ou superior ao modelo *HPC-Coder-V2-6.7b*, apesar de possuírem quase cinco vezes menos parâmetros.

Ao avaliar o desempenho dos modelos quanto à geração de código CUDA funcionalmente correto, ou seja, sob a perspectiva da métrica $pass@k$, observamos na Figura 5.7 que o modelo *HPC-Coder-V2-6.7b* apresenta a melhor taxa média de acertos para $k \leq 5$. Ainda assim, já em $k = 5$, o modelo *GPPD-HPC-CUDA-Coder-full-dataset* aproxima-se consideravelmente desse desempenho, indicando que, apesar da diferença significativa no número de parâmetros, o refinamento especializado em CUDA reduz de forma substancial essa distância.

Entretanto, a média geral tende a mascarar comportamentos específicos entre os tipos de problemas avaliados. Ao analisarmos o cenário mais restritivo, considerando apenas $k = 1$, ilustrado na Figura 5.8, torna-se evidente que o modelo *HPC-Coder-V2* se destaca principalmente devido ao desempenho elevado em problemas como *histogram* e *stencil*, que elevam sua média global. Por outro lado, os modelos *GPPD-HPC-CUDA-Coder* apresentam desempenho semelhante, e em alguns casos superior, ao modelo de maior porte, notadamente em tarefas do tipo *search*, evidenciando que o refinamento direcionado ao domínio de CUDA permite competir de forma eficaz mesmo frente a modelos substancialmente maiores.

Um ponto relevante a ser destacado é que todos os modelos apresentam dificulda-

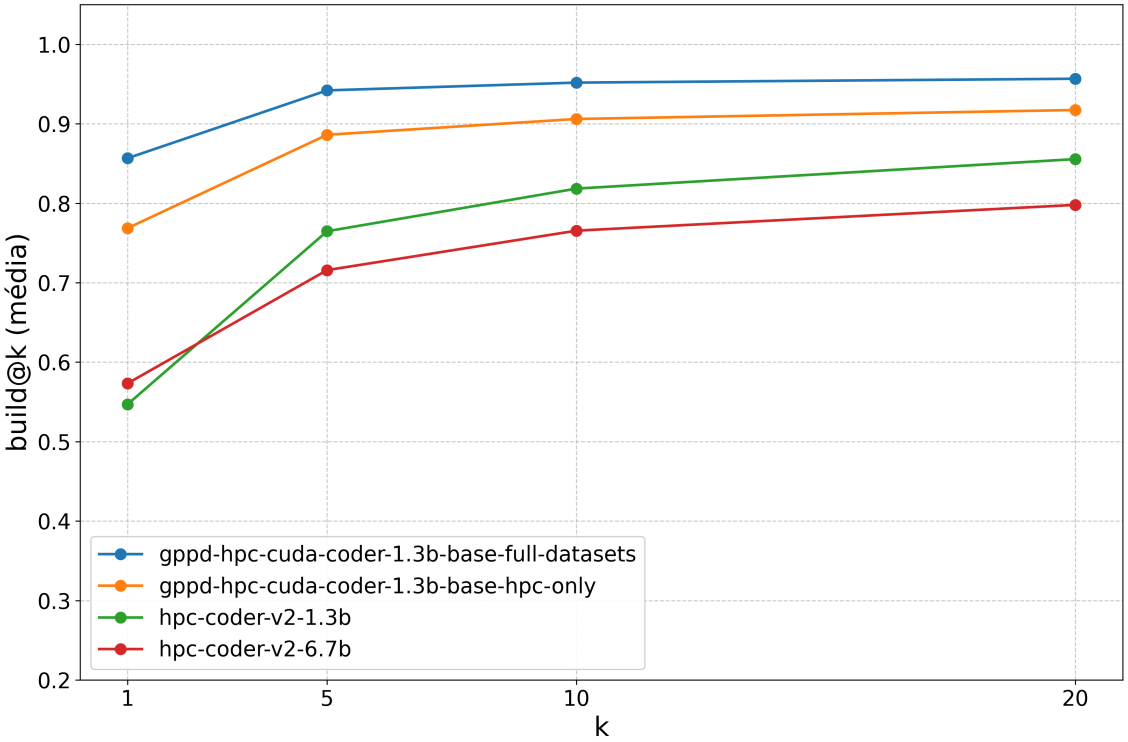


Figura 5.5: Progressão da média de $build@k$ entre os modelos refinados

des em produzir código funcional para problemas das categorias *geometry*, *reduce*, *scan* e *sparse_la*, mesmo quando exibem valores elevados de $build@1$ nessas mesmas tarefas. Esse comportamento evidencia que a capacidade de compilar o código não implica, necessariamente, em sua correção funcional.

5.3 Análise de eficiência computacional

A avaliação de desempenho apresentada nas seções anteriores evidencia que o refinamento especializado em código CUDA permite que o modelo *GPPD-HPC-CUDA-Coder-1.3b* alcance resultados comparáveis, e em diversos cenários superiores, aos modelos da linha *HPC-Coder-V2*, mesmo quando estes possuem uma quantidade significativamente maior de parâmetros e foram treinados em infraestruturas computacionais muito mais robustas. Essa constatação torna essencial analisar também o custo computacional associado ao desenvolvimento de cada modelo, permitindo contextualizar a eficiência do processo de refinamento empregado neste trabalho.

Enquanto os modelos *HPC-Coder-V2-1.3b* e *HPC-Coder-V2-6.7b* foram treinados em ambientes compostos por dois e quatro nós equipados com GPUs A100 de 80GB, respectivamente, totalizando aproximadamente 240GB de memória dedicada ao treina-

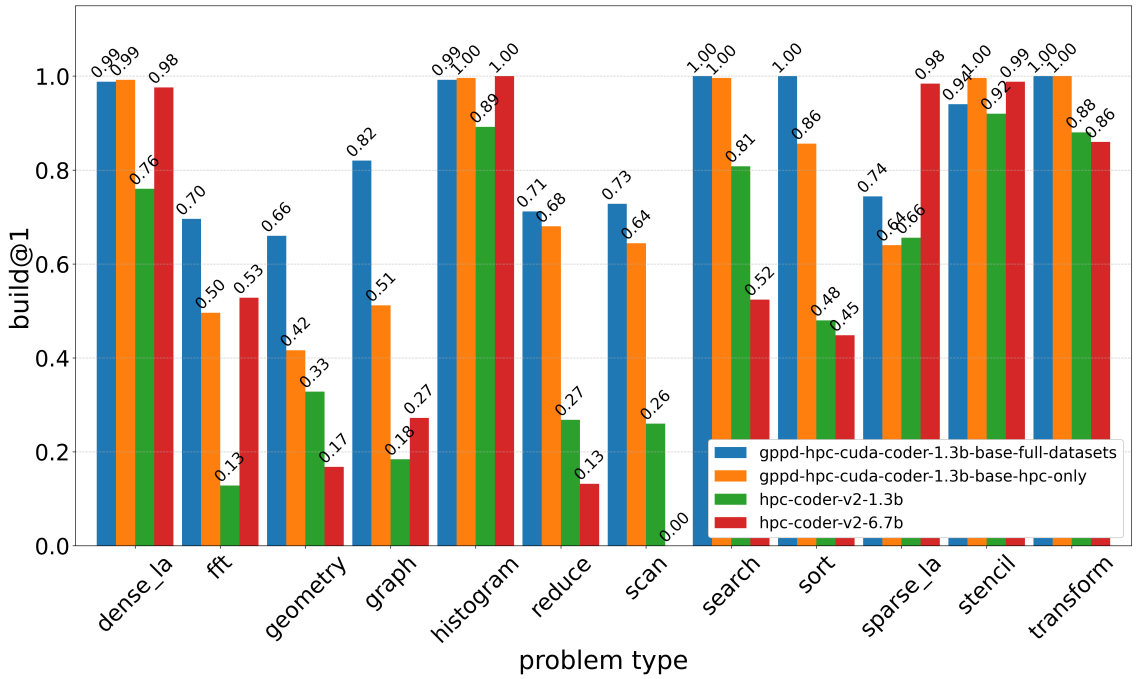


Figura 5.6: Resultados de $build@1$ por tipo de problema entre os modelos refinados

mento, o refinamento do modelo *GPPD-HPC-CUDA-Coder-1.3b* foi realizado utilizando apenas uma GPU RTX 4090 com 24GB de memória. Essa discrepância evidencia não apenas a diferença de escala entre as infraestruturas necessárias, mas também o contraste entre os custos energéticos e operacionais envolvidos nos dois processos.

Os resultados obtidos reforçam um ponto destacado por Belcak et al. (2025): modelos menores, quando adequadamente adaptados a um domínio específico, podem oferecer desempenho competitivo em relação a modelos de grande porte, ao mesmo tempo em que apresentam maior sustentabilidade, reprodutibilidade e acessibilidade. O caso do *GPPD-HPC-CUDA-Coder* exemplifica esse princípio, demonstrando que um processo de refinamento direcionado e eficiente é capaz de produzir um modelo especializado de alta qualidade sem a necessidade de infraestruturas de grande escala.

Dessa forma, o *GPPD-HPC-CUDA-Coder* se destaca não apenas pelos resultados técnicos apresentados, mas também por sua viabilidade prática. Ele representa uma alternativa promissora para o desenvolvimento de modelos especializados em HPC, conciliando alto desempenho com baixo custo computacional e viabilizando a adoção de abordagens baseadas em *Small Language Models* no contexto da geração automática de código paralelo.

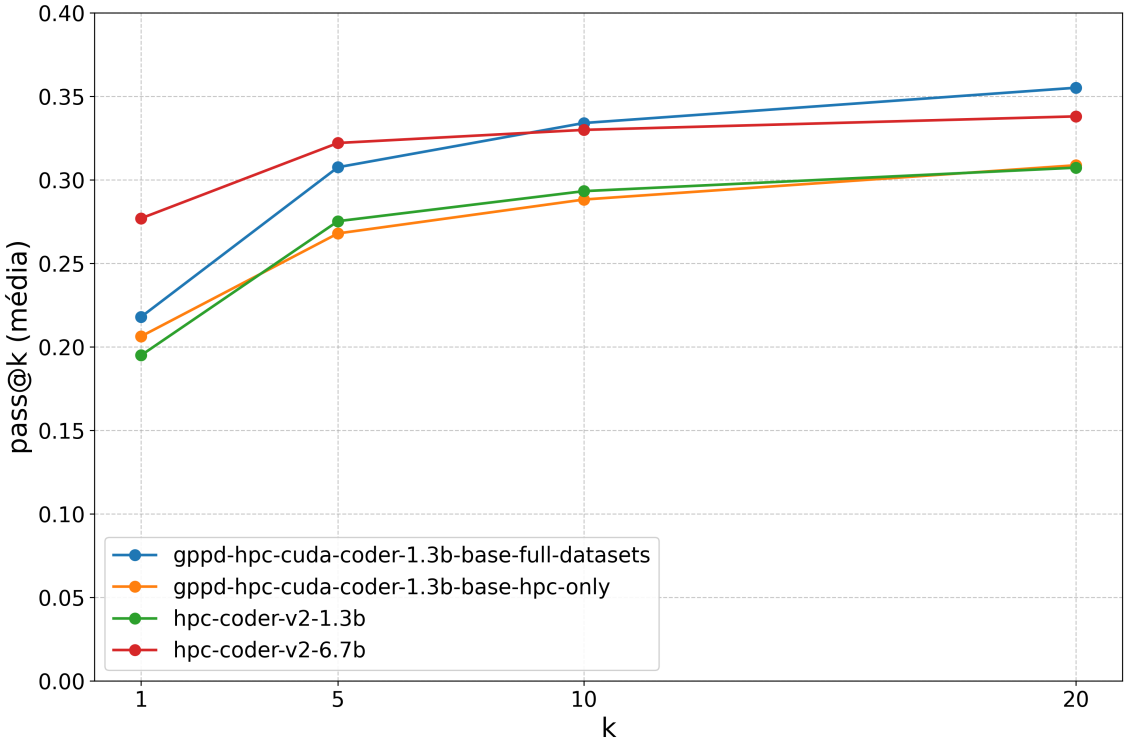


Figura 5.7: Progressão da média de $pass@k$ entre todos modelos

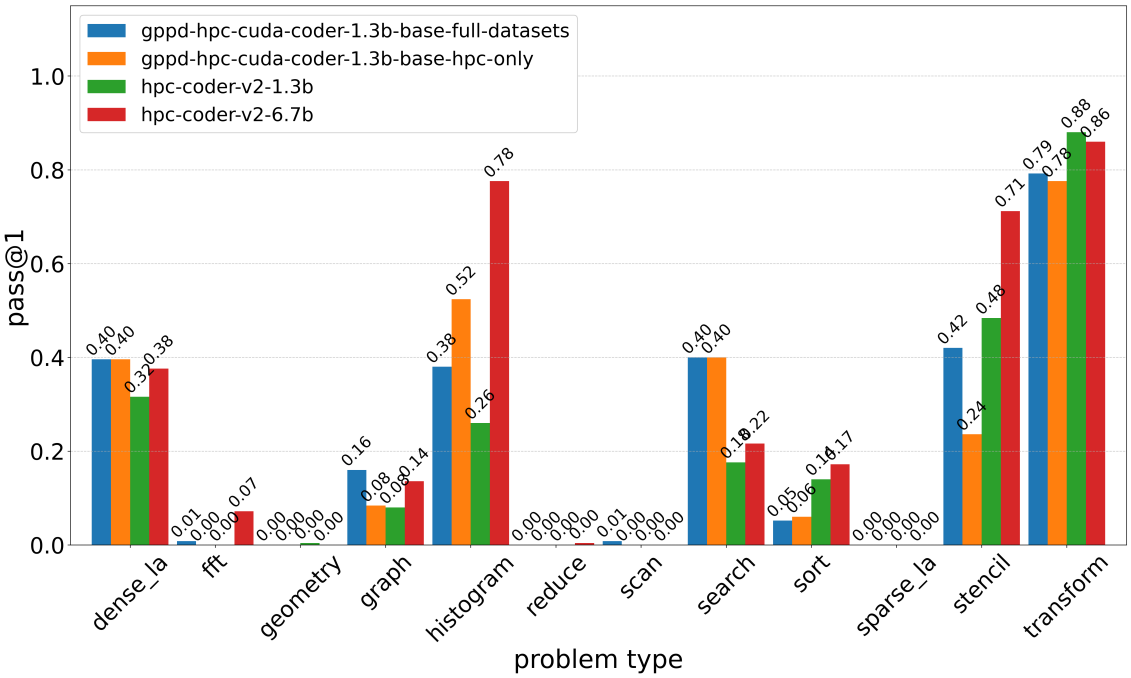


Figura 5.8: Resultado de $pass@1$ por tipo de problema entre os modelos refinados

6 CONCLUSÃO

Este trabalho teve como objetivo investigar se um *Small Language Model* (SLM), quando refinado com um *dataset* especializado em CUDA, seria capaz de gerar código paralelo com desempenho competitivo frente a modelos maiores e generalistas. Para isso, construímos um conjunto de dados dedicado, realizamos o processo de *fine-tuning*, avaliamos o modelo utilizando o *benchmark* ParEval e comparamos seus resultados com modelos de referência no domínio de código paralelo.

Os resultados demonstram de forma consistente que a especialização voltada à geração de código CUDA é capaz de elevar substancialmente o desempenho de modelos de pequeno porte, permitindo-lhes competir com modelos de estado da arte significativamente maiores. A utilização do ParEval possibilitou uma avaliação completa, contemplando tanto a capacidade de compilação (*build@k*) quanto a correção funcional (*pass@k*), elementos fundamentais para medir a qualidade do código paralelo produzido.

A análise entre os modelos da família *GPPD-HPC-CUDA-Coder* evidenciou que, embora o modelo base *DeepSeek-Coder-1.3b* obtenha ligeira vantagem na métrica de compilabilidade, os modelos refinados superam-no de maneira consistente quando a avaliação se concentra na correção funcional. Esse comportamento indica que o refinamento especializado promove maior robustez semântica na geração de soluções corretas. Observou-se ainda que a adição de dados não específicos de HPC traz benefícios restritos a certas categorias, não resultando em melhorias amplas.

Na comparação com os modelos *HPC-Coder-V2*, observou-se que a especialização em CUDA confere aos modelos *GPPD* desempenho superior em *build@k*, mesmo frente a modelos com número de parâmetros até cinco vezes maior. Em relação à métrica *pass@k*, o modelo *HPC-Coder-V2-6.7b* apresenta melhor média geral, mas o *GPPD-HPC-CUDA-Coder-full-dataset* rapidamente se aproxima desse desempenho para valores moderados de k. Já na análise de $pass@1$, os modelos especializados mostram competitividade — e, em algumas categorias, desempenho superior — evidenciando que a adaptação ao domínio CUDA gera melhorias reais, mesmo diante de arquiteturas significativamente menores.

Outro ponto relevante é a dissociação observada entre código compilável e código funcionalmente correto. Todas as famílias de modelos apresentaram dificuldades com categorias como *geometry*, *reduce*, *scan* e *sparse_la*, ainda que alcançassem altas taxas de *build@1*. Isso reforça a necessidade de abordagens mais profundas voltadas à

compreensão semântica dos padrões paralelos envolvidos.

A análise de eficiência computacional destaca a segunda grande contribuição deste trabalho. Enquanto a linha *HPC-Coder-V2* requer infraestrutura com múltiplas GPUs A100 e centenas de gigabytes de memória, todo o refinamento do *GPPD-HPC-CUDA-Coder-1.3b* foi realizado em uma única GPU RTX 4090. A discrepância entre recursos utilizados e desempenho alcançado demonstra a viabilidade prática do uso de SLMs especializados, alinhando-se às discussões recentes sobre as vantagens de *Small Language Models* para agentes especializados.

Em síntese, o *GPPD-HPC-CUDA-Coder* apresenta não apenas competitividade técnica, mas também eficiência operacional, acessibilidade e forte potencial de reprodutibilidade. Os resultados obtidos reforçam que a especialização guiada por domínio constitui uma alternativa sólida ao aumento indiscriminado de escala dos modelos, oferecendo um caminho promissor para o desenvolvimento de ferramentas de apoio à programação paralela. Como direções futuras, destacam-se a ampliação de *datasets* especializados, a incorporação de métodos de treinamento voltados à compreensão estrutural de padrões paralelos e a avaliação do modelo em cenários reais de desenvolvimento de aplicações HPC.

REFERÊNCIAS

ACHIAM, J. et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

BELCAK, P. et al. Small language models are the future of agentic ai. *arXiv preprint arXiv:2506.02153*, 2025.

CHANDY, K. M. Parallel program design. In: *Opportunities and Constraints of Parallel Computing*. [S.l.]: Springer, 1989. p. 21–24.

CHATURVEDI, A. et al. Hpc-coder-v2: Studying code llms across low-resource parallel languages. *arXiv preprint arXiv:2412.15178*, 2024.

CHEN, L. et al. Ompgpt: A generative pre-trained transformer model for openmp. In: SPRINGER. *European Conference on Parallel Processing*. [S.l.], 2024. p. 121–134.

CHEN, M. et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

DIAS, J. V. M.; BECK, A. C. S.; LORENZON, A. F. Explorando modelos gpt na geraçao de código paralelo para aplicaçoes de computaçao stencil. In: SBC. *Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD)*. [S.l.], 2024. p. 348–359.

GUO, D. et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

KADOSH, T. et al. Monocoder: Domain-specific code language model for hpc codes and tasks. In: IEEE. *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. [S.l.], 2024. p. 1–7.

LARA, P. V. et al. Chathpc: Building the foundations for a productive and trustworthy ai-assisted hpc ecosystem. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. [S.l.: s.n.], 2025. p. 458–474.

LEI, B. et al. Creating a dataset for high-performance computing code translation using llms: A bridge between openmp fortran and c++. In: *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. [S.l.: s.n.], 2023. p. 1–7.

LOZHKOV, A. et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.

LUO, Z. et al. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023.

MUNLEY, C.; JARMUSCH, A.; CHANDRASEKARAN, S. Llm4vv: Developing llm-driven testsuite for compiler validation. *Future Generation Computer Systems*, v. 160, p. 1–13, 2024. ISSN 0167-739X. Available from Internet: <https://www.sciencedirect.com/science/article/pii/S0167739X24002449>.

NICHOLS, D. et al. Can large language models write parallel code? In: *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*. [S.l.: s.n.], 2024. p. 281–294.

NICHOLS, D. et al. Hpc-coder: Modeling parallel programs using large language models. In: PROMETEUS GMBH. *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*. [S.l.], 2024. p. 1–12.

NVIDIA Corporation. *CUDA C++ Programming Guide*. [S.l.], 2025. Acesso em: 12 nov. 2025. Available from Internet: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.

ROZIERE, B. et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

SHANI, G. I. *Survey Reveals AI's Impact on the Developer Experience*. 2024. <https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/>. Acesso em: 17 maio 2025.

VASWANI, A. et al. Attention is all you need. *Advances in neural information processing systems*, v. 30, 2017.

WEI, Y. et al. Magicoder: Empowering code generation with oss-instruct. *arXiv preprint arXiv:2312.02120*, 2023.

ZHOU, A.; HAWKINS, L.; GENTINE, P. Proof-of-concept: using chatgpt to translate and modernize an earth system model from fortran to python/jax. *arXiv preprint arXiv:2405.00018*, 2024.

ZHU, Q. et al. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931*, 2024.